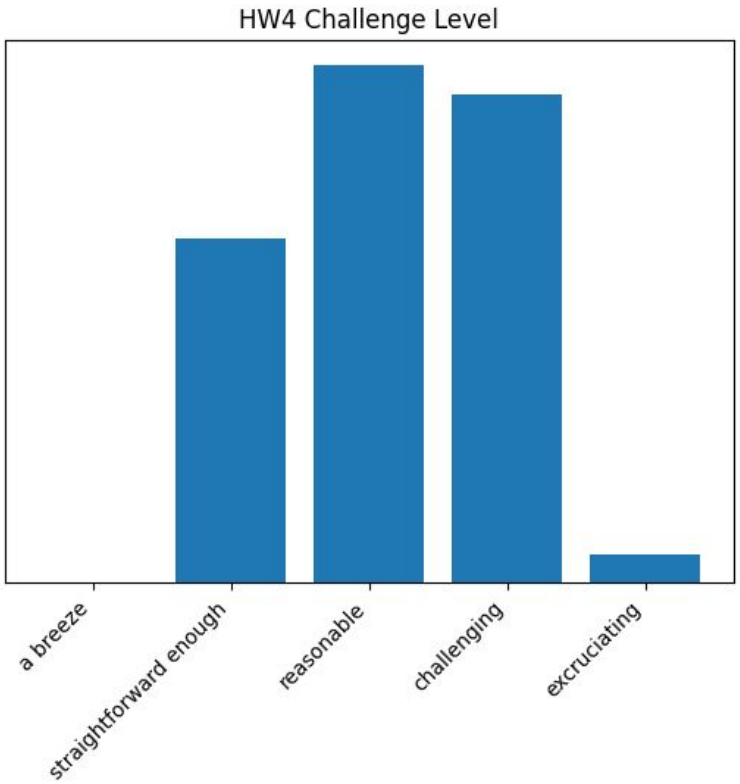
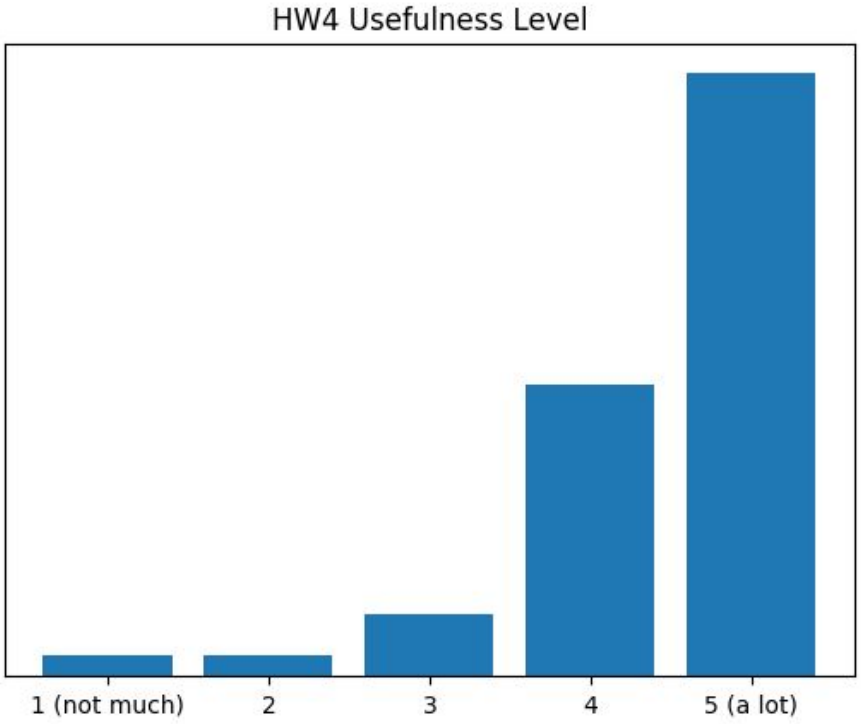
CSE 493L Section 6

Concurrent Data Structures in Rust

Looking Back on Homework 4





Homework 6: Due Dates

- **Monday, February 16th:** test case - hard deadline
- **Wednesday, February 18th**: code, questions - 2 day grace period

Implementing thread safe Cuckoo Hash!

Check-in

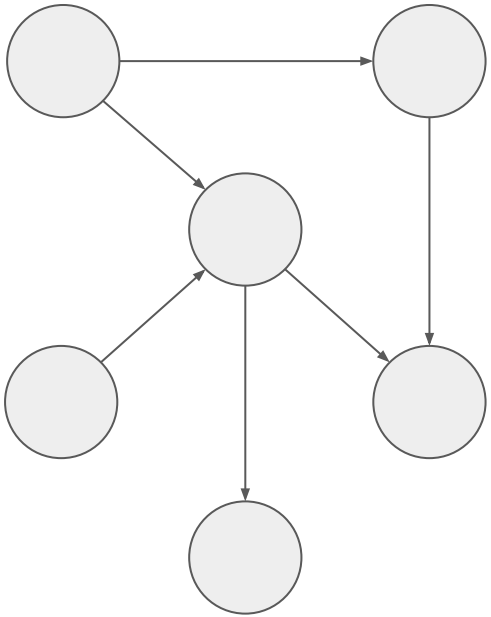
- Before we get started: *any questions?*

Designing a Graph

Let’s say we want (for whatever reason) to create a linked graph structure.

Trees are “nice” because of their ownership structure, each node only has one owner.

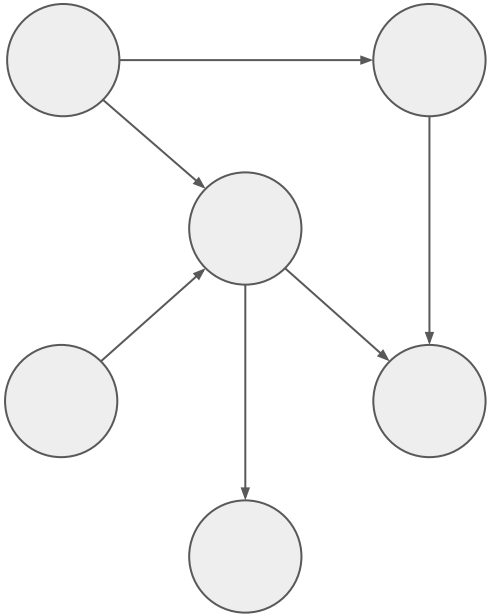
Graphs are mean because each node can have multiple owners?!



Designing a Graph

Let's say we want (for whatever reason) to create a linked graph structure using the following:

```
struct Node<T> {
    my_data: T,
    neighbors: Vec<???>
}
```

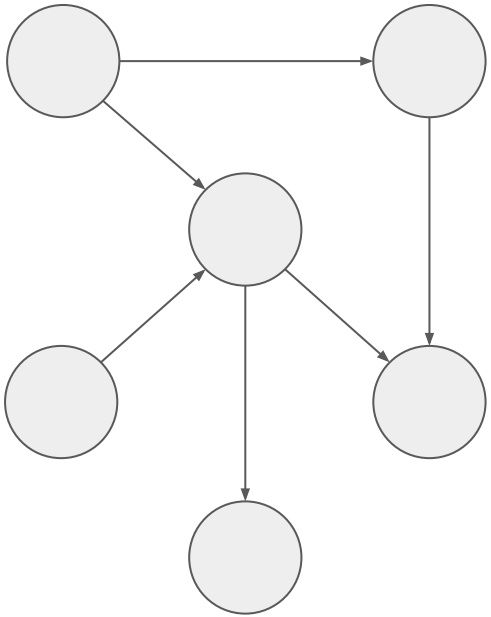


What should go in ???

Designing a Graph

Let's say we want (for whatever reason) to create a linked graph structure using the following:

```
struct Node<T> {
    my_data: T,
    neighbors: Vec<Box<Node>>
}
```

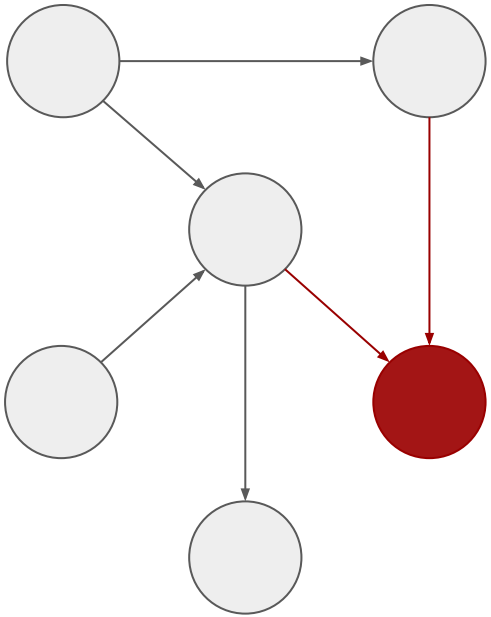


Trees used `Box`! Maybe we can use a `Box`?

Designing a Graph

Let's say we want (for whatever reason) to create a linked graph structure using the following:

```
struct Node<T> {
    my_data: T,
    neighbors: Vec<Box<Node>>
}
```



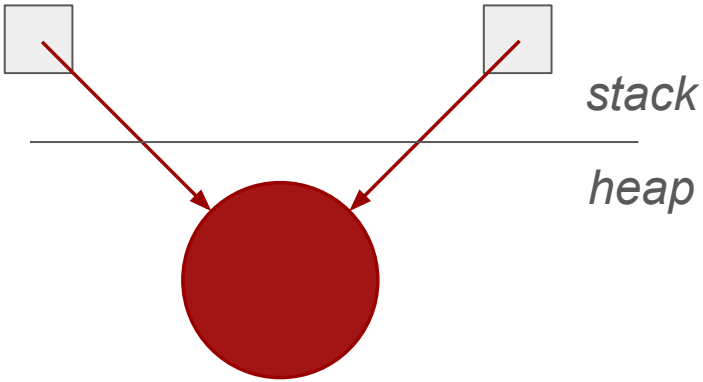
Trees used `Box`! Maybe we can use a `Box`?

… but then wouldn't this node have two owners?

Designing a Graph

Let’s abstract a little…

The problem comes from needing two owners of the same data, while the language only permits one.

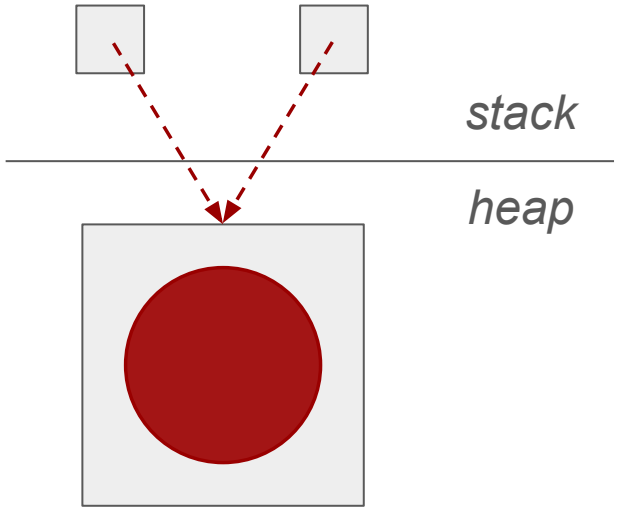


Designing a Graph

Let's abstract a little…

Let's add a layer of indirection, and use a *raw memory pointer* (that's non-null).

- These pointers don't carry ownership.
- We'll implement Deref on the stack-level data type to allow for `*ptr` to return `&data`



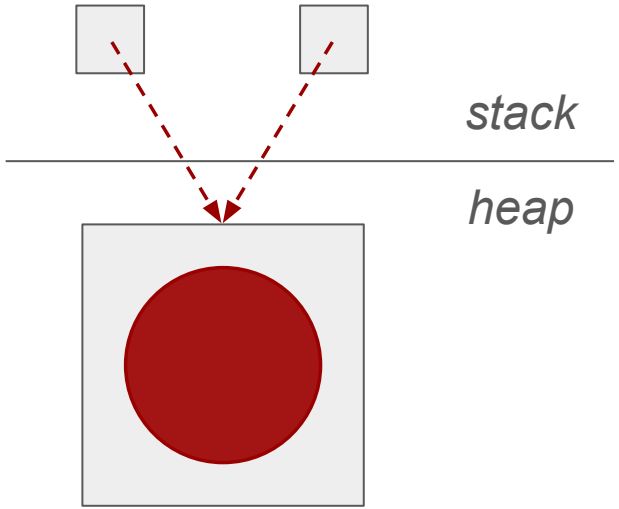
Designing a Graph

Let's abstract a little…

Let's add a layer of indirection, and use a *raw memory pointer* (that's non-null).

- These pointers don't carry ownership.
- We'll implement Deref on the stack-level data type to allow for `*ptr` to return `&data`

If they don't carry ownership, how does anything behind that pointer get cleaned up???



Designing a Graph

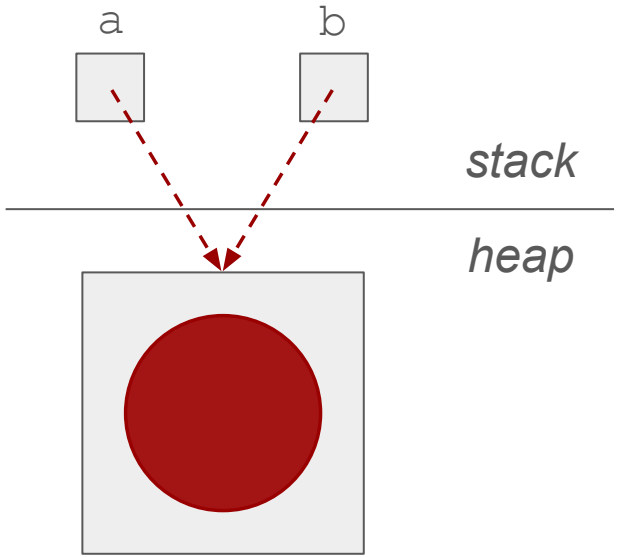
Let's abstract a little…

Let's add a layer of indirection, and use a *raw memory pointer* (that's non-null).

- These pointers don't carry ownership.
- We'll implement Deref on the stack-level data type to allow for `*ptr` to return `&data`

If they don't carry ownership, how does anything behind that pointer get cleaned up???

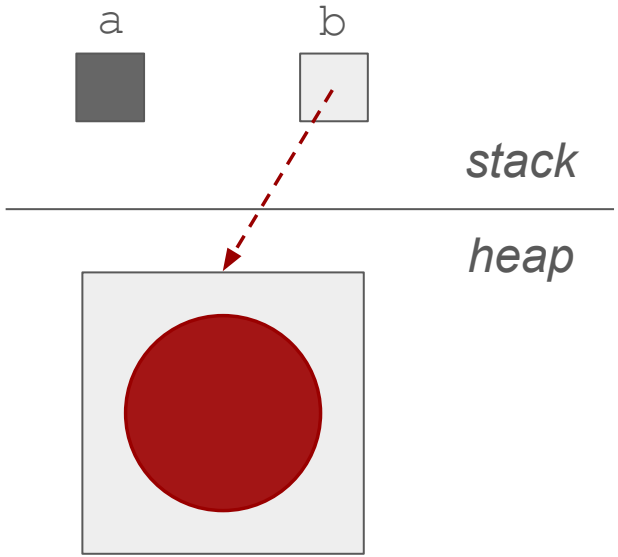
- *Does `drop(a)` do anything???*



Designing a Graph

`drop(a)` could just invalidate using `a`.

Can we deallocate the node?



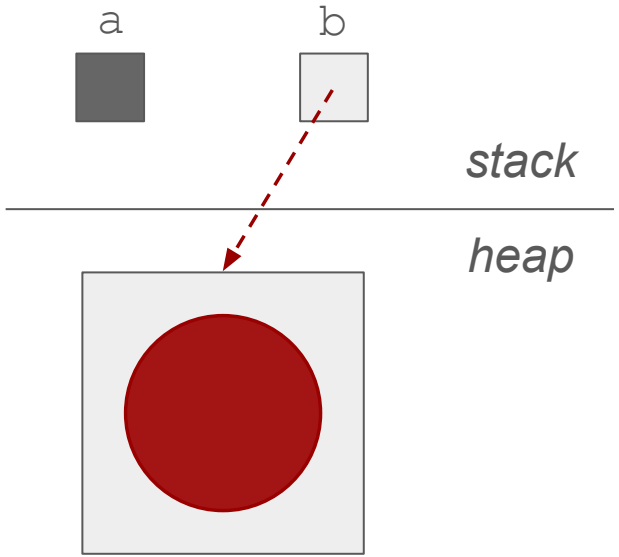
Designing a Graph

`drop(a)` could just invalidate using `a`.

Can we deallocate the node?

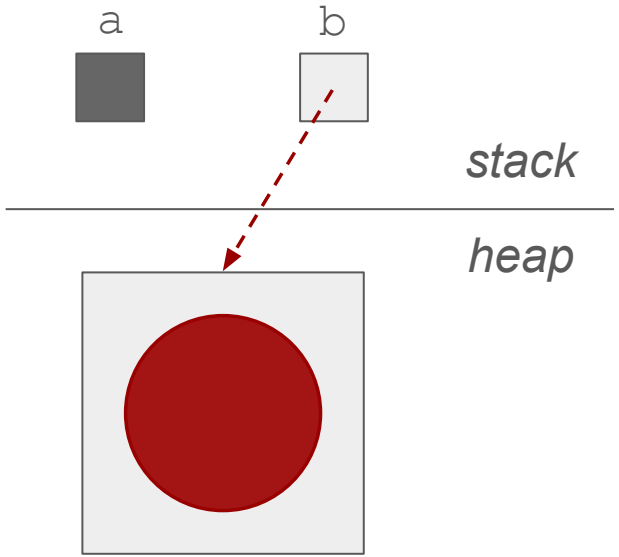
- No! `b` can still access it!

So when can we deallocate the node?



Designing a Graph

`drop(a)` could just invalidate using `a`.



Can we deallocate the node?

- No! `b` can still access it!

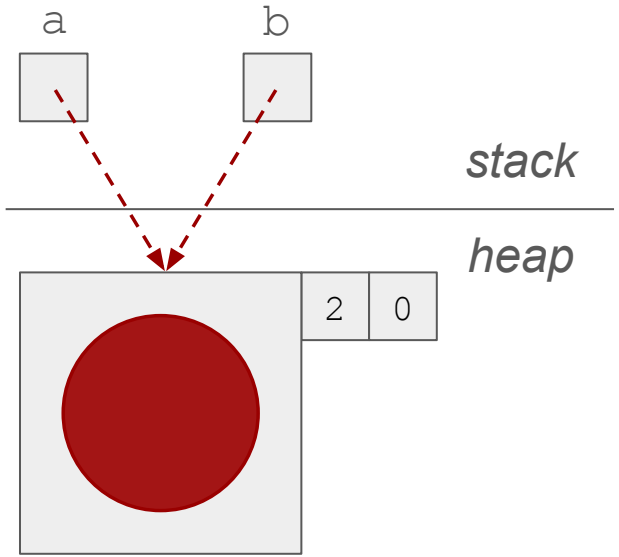
So when can we deallocate the node?

- When every pointer to the heap structure is dropped (a reference count!)
- We need to keep track of some metadata.

Designing a Graph

Let’s add a strong count and a weak count.

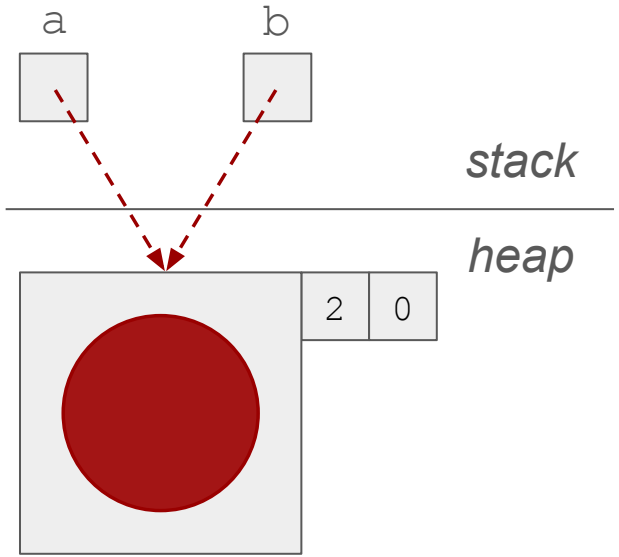
- Weak count to allow for breaking circular patterns. Don’t worry about it for now.



Designing a Graph

Let's add a strong count and a weak count.

- Weak count to allow for breaking circular patterns. Don't worry about it for now.

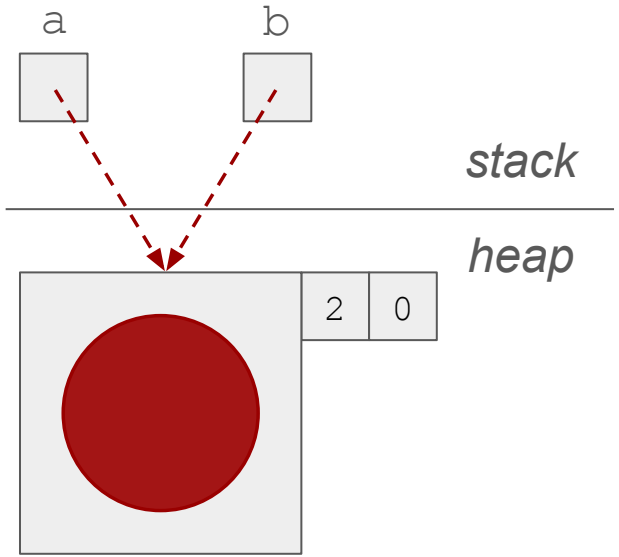


`drop(a)` can now just use the pointer to decrement the strong count, and conditionally free resources.

Conceptually, `a` and `b` now "share" ownership!

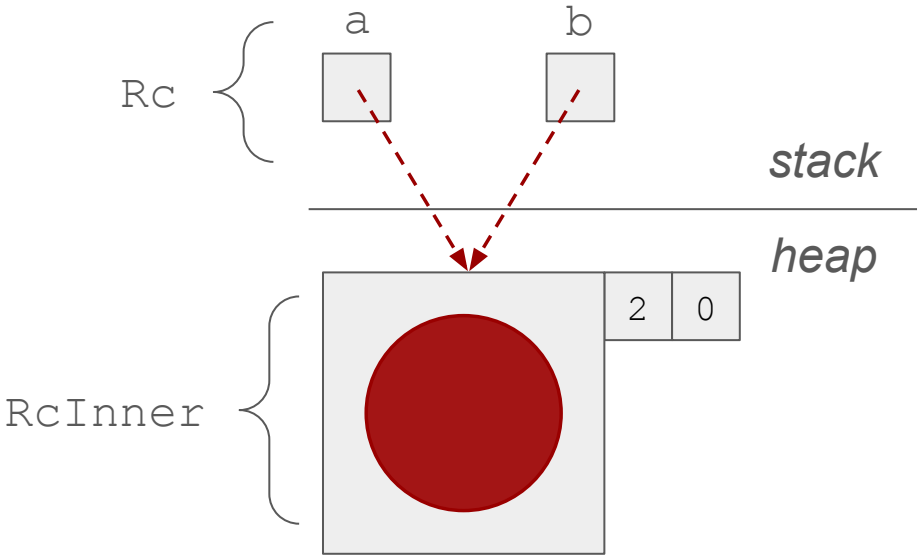
Designing a Graph

We just designed an `Rc<T>`!



Designing a Graph

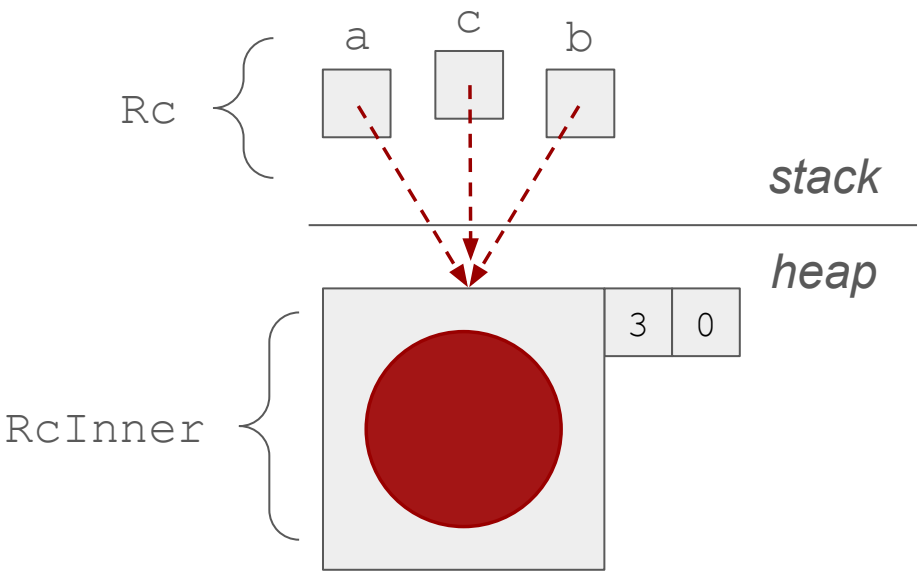
We just designed an `Rc<T>`!



Designing a Graph

We just designed an `Rc<T>`!

`c = a.clone()` will return a new Rc, incrementing the strong count.

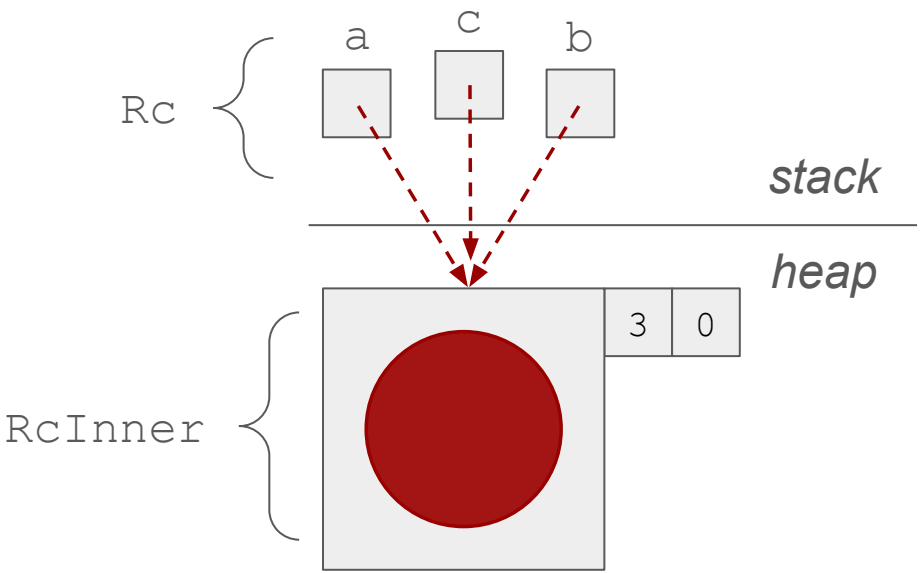


Designing a Graph

We just designed an `Rc<T>`!

`c = a.clone()` will return a new Rc, incrementing the strong count.

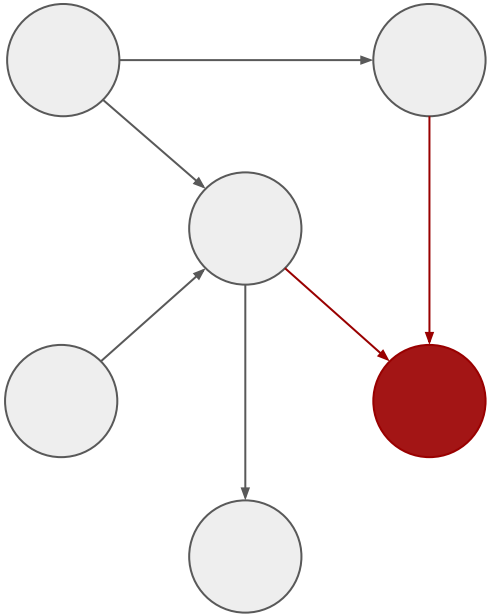
`drop(c)` will decrement the strong count, until it's zero, in which case the associated `T` is dropped.



Designing a Graph

Let's say we want (for whatever reason) to create a linked graph structure using the following:

```
struct Node<T> {
    my_data: T,
    neighbors: Vec<Rc<Node>>
}
```

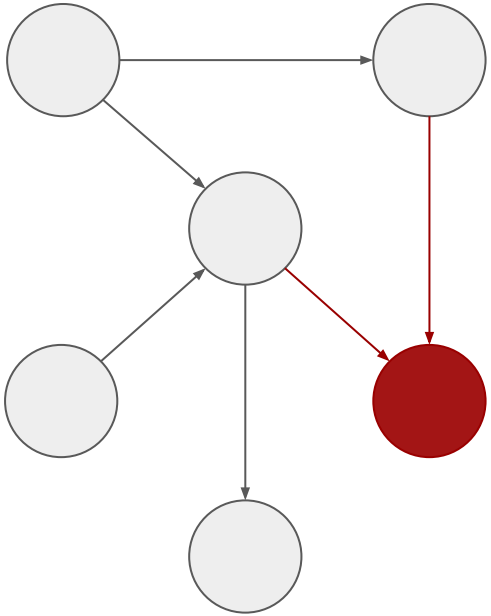


This fixes our multiple-owner problem!

Designing a Graph

Let's say we want (for whatever reason) to create a linked graph structure using the following:

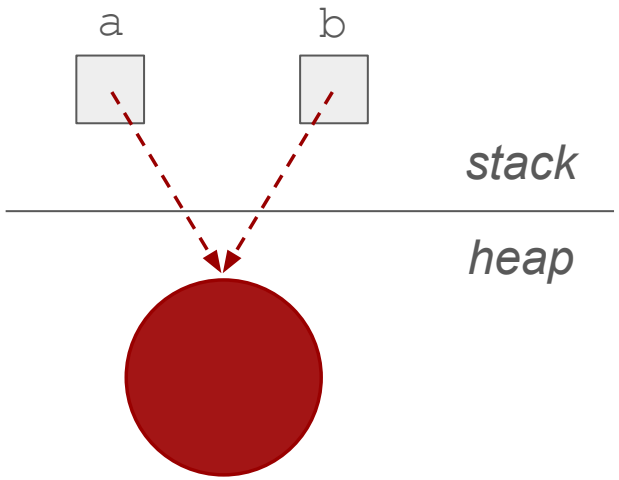
```
struct Node<T> {
    my_data: T,
    neighbors: Vec<Rc<Node>>
}
```



`Rc` is single-thread safe, `Arc` is multi-thread safe.

Designing a Graph

Abstracting away `RcInner`, we can think of `Rc` as:

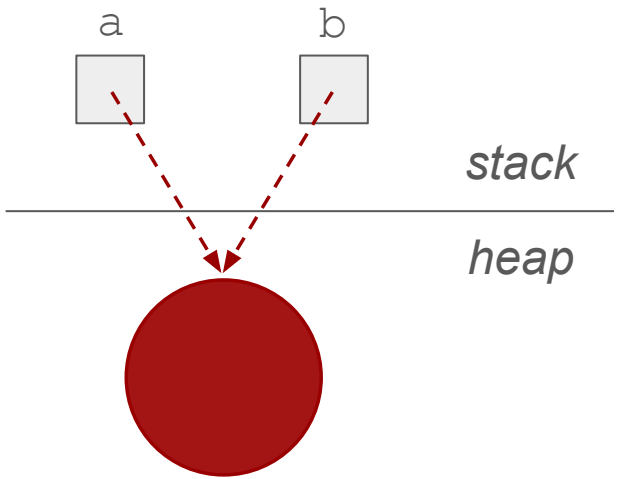


Designing a Graph

Abstracting away `RcInner`, we can think of `Rc` as:

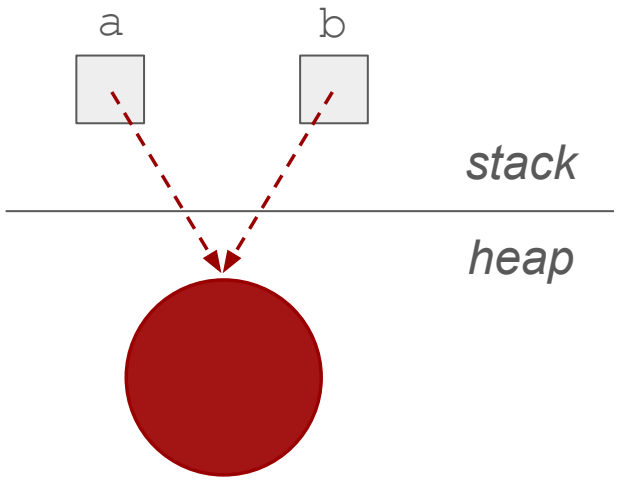
Can we safely allow `*a` to return `&mut data`?

- This would mean implementing `DerefMut`



Designing a Graph

Abstracting away `RcInner`, we can think of `Rc` as:



Can we safely allow `*a` to return `&mut data`?

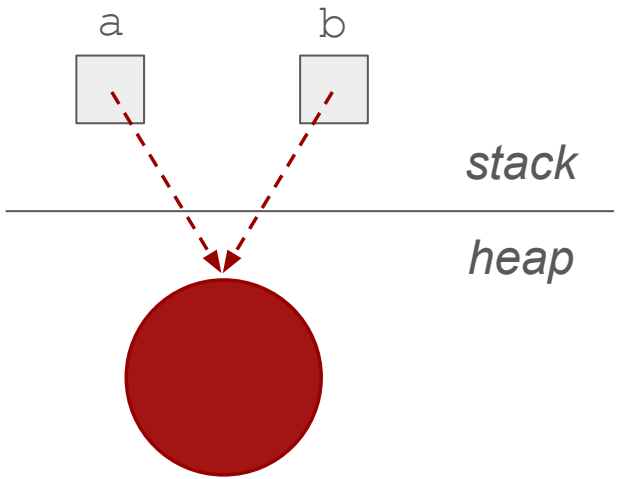
- No! a and b have no knowledge of each other, and therefore that would allow things like:

```
fn foo(a: Rc<Node>, b: Rc<Node>) {
    let a_ref: &mut Node = *a;
    let b_ref: &mut Node = *b;
}
```

The compiler can't tell that `a` and `b` are the same!

Designing a Graph

Abstracting away `RcInner`, we can think of `Rc` as:



Can we safely allow `*a` to return `&mut data`?

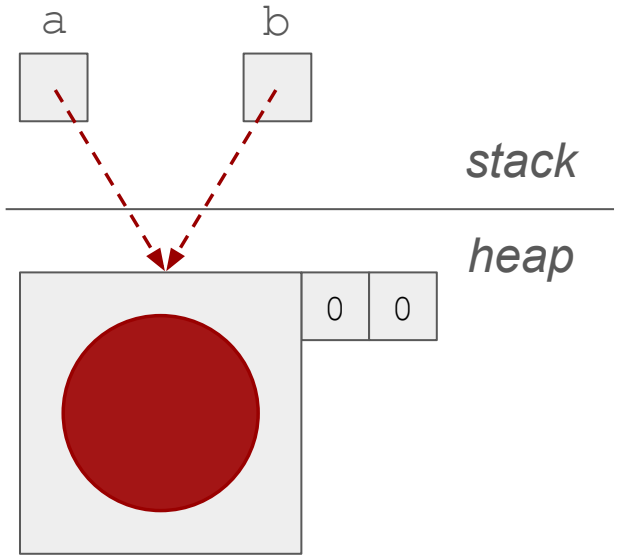
- No!

But wow it would ***suck*** if we were never able to mutate the contents of a `Node`!

Designing a Graph

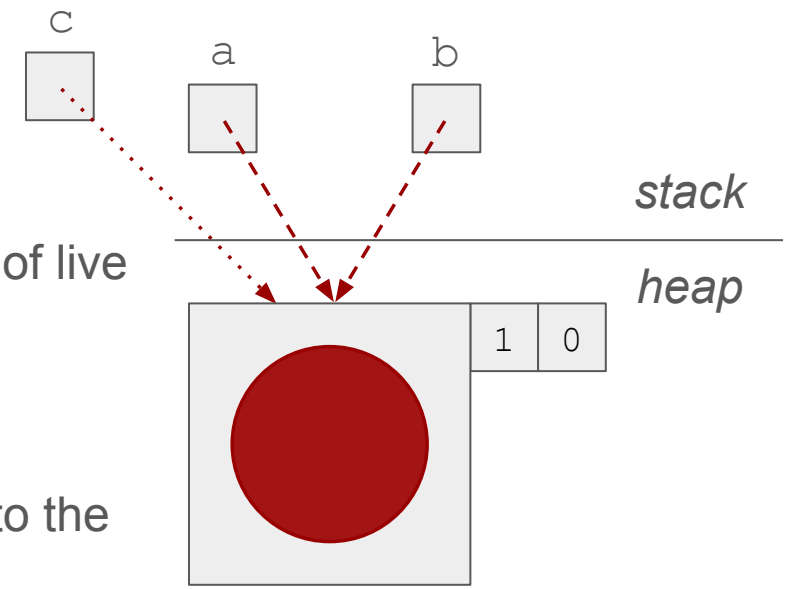
Let's add another layer of indirection.

Same structure as Rc, but tracking the number of live shared / mutable references.



Designing a Graph

Let's add another layer of indirection.



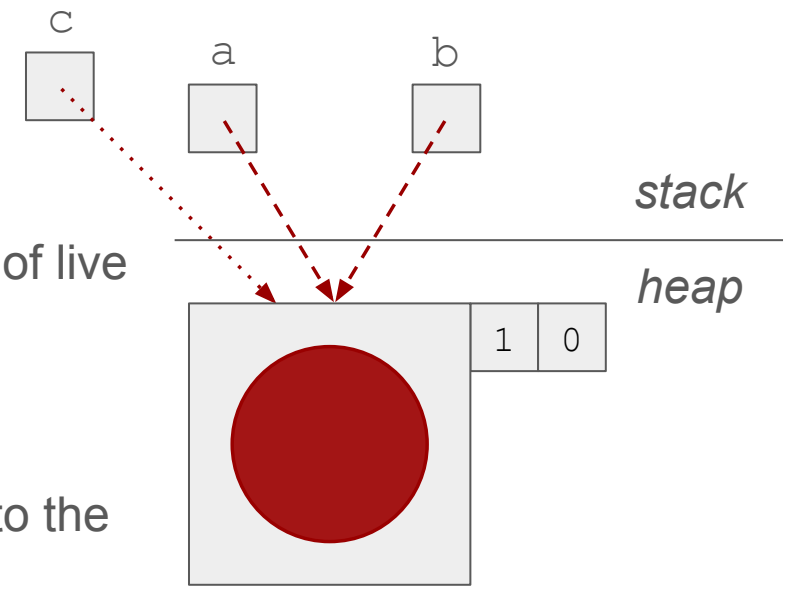
Same structure as Rc, but tracking the number of live shared / mutable references.

`c = a.borrow()` will get a shared reference to the data, and increment the appropriate count.

- `c` here is a shared reference to the wrapped data

Designing a Graph

Let's add another layer of indirection.



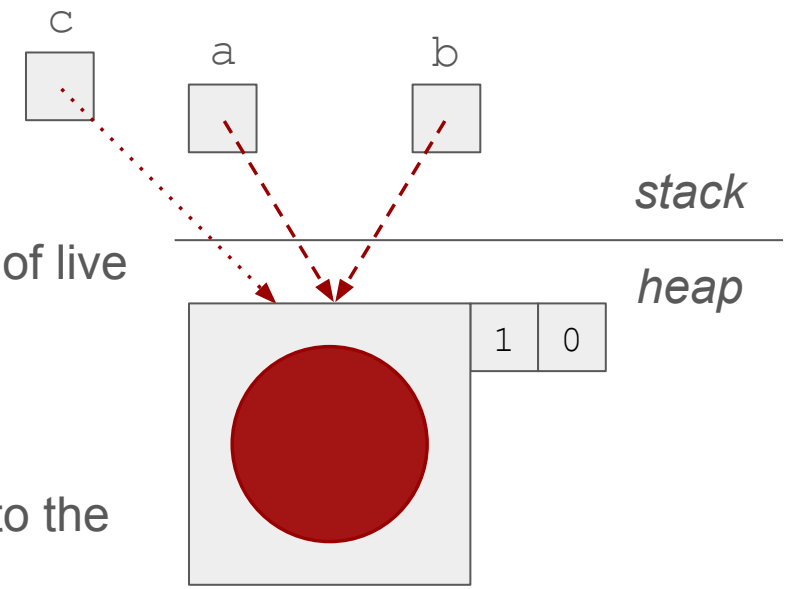
Same structure as Rc, but tracking the number of live shared / mutable references.

`c = a.borrow()` will get a shared reference to the data, and increment the appropriate count.

- Can we now execute `d = b.borrow_mut()`?

Designing a Graph

Let's add another layer of indirection.



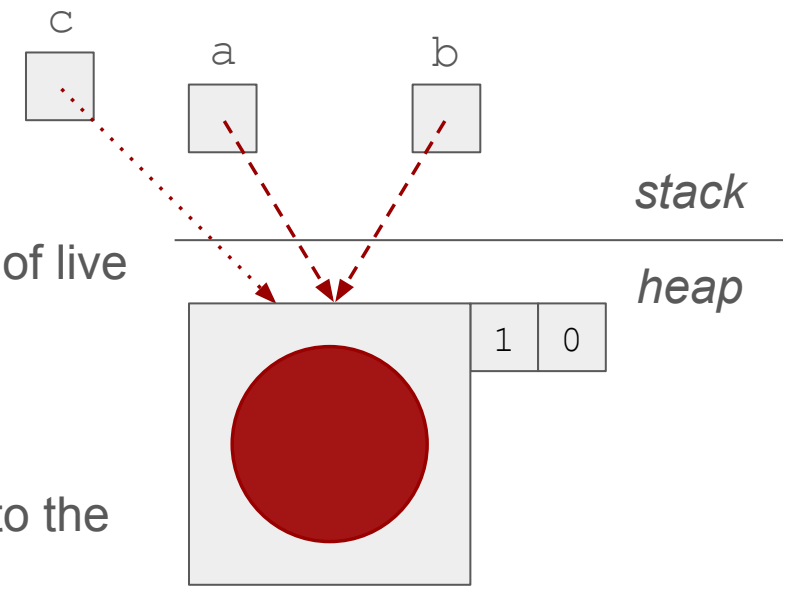
Same structure as Rc, but tracking the number of live shared / mutable references.

`c = a.borrow()` will get a shared reference to the data, and increment the appropriate count.

- Can we now execute `d = b.borrow_mut()`?
- No! The shared count is `>0`! That would violate borrowing rules!

Designing a Graph

Let's add another layer of indirection.



Same structure as Rc, but tracking the number of live shared / mutable references.

`c = a.borrow()` will get a shared reference to the data, and increment the appropriate count.

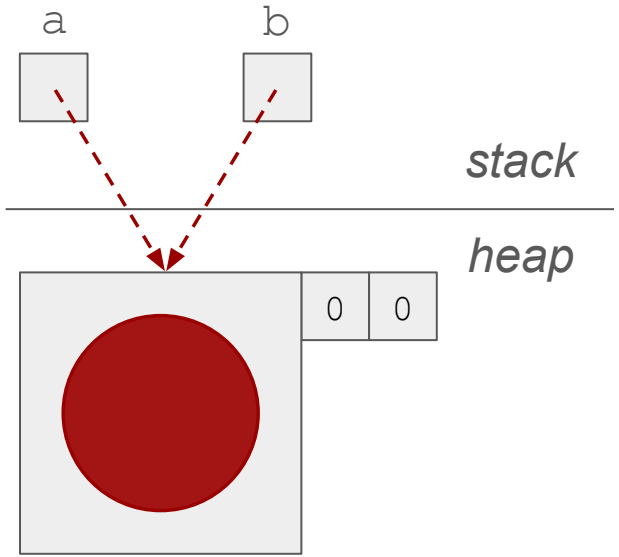
- Can we now execute `d = b.borrow_mut()`?
- Can we now execute `d = b.borrow()`?

Designing a Graph

Let's add another layer of indirection.

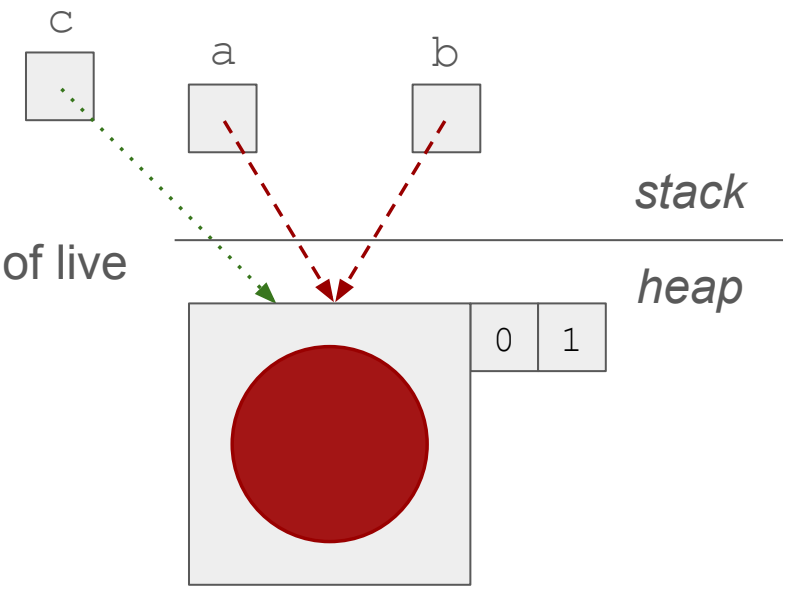
Same structure as Rc, but tracking the number of live shared / mutable references.

Can we execute `c = a.borrow_mut()`?



Designing a Graph

Let's add another layer of indirection.



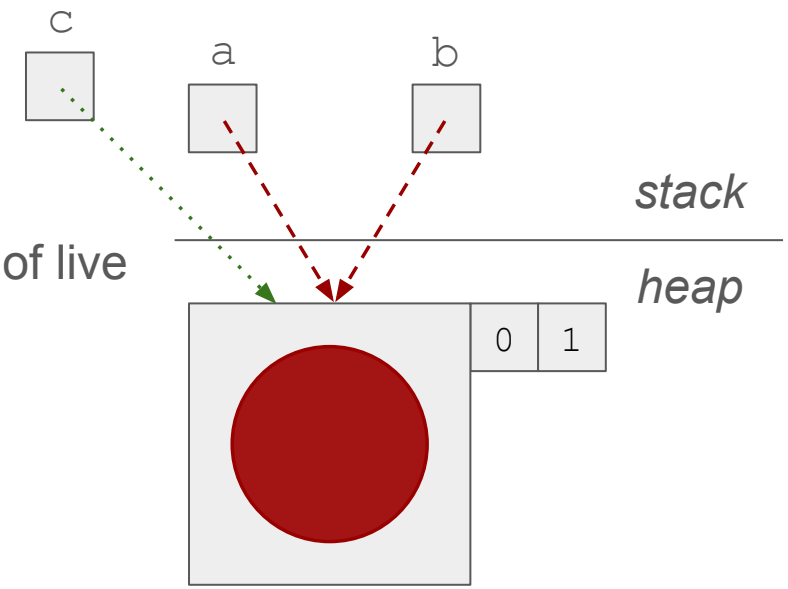
Same structure as Rc, but tracking the number of live shared / mutable references.

Can we execute `c = a.borrow_mut()`?

- Yup! No shared or mutable references exist yet!

Designing a Graph

Let's add another layer of indirection.



Same structure as Rc, but tracking the number of live shared / mutable references.

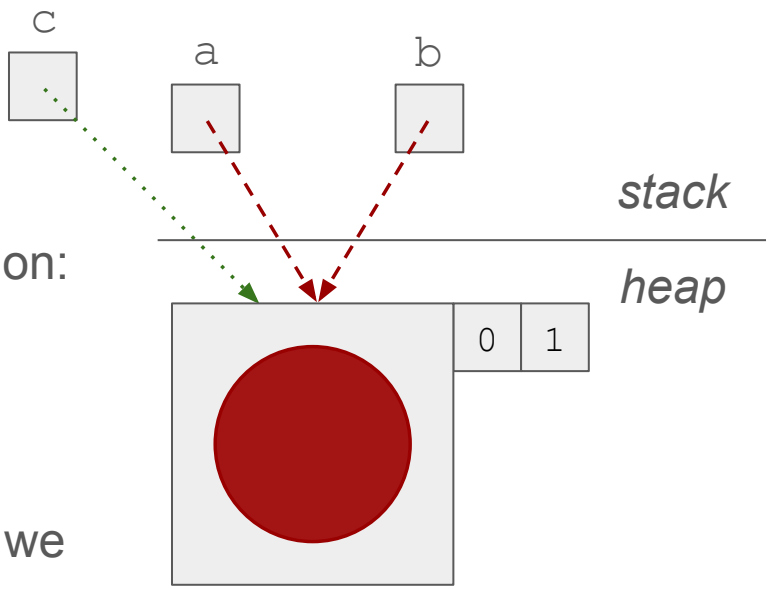
Can we execute `c = a.borrow_mut()`?

- Yup! No shared or mutable references exist yet!

Note: this *defers* borrow checking to runtime!

Designing a Graph

We just designed a `RefCell<T>`



Formally, a `RefCell` essentially defines a function:

```
&RefCell<T> -> &mut T
```

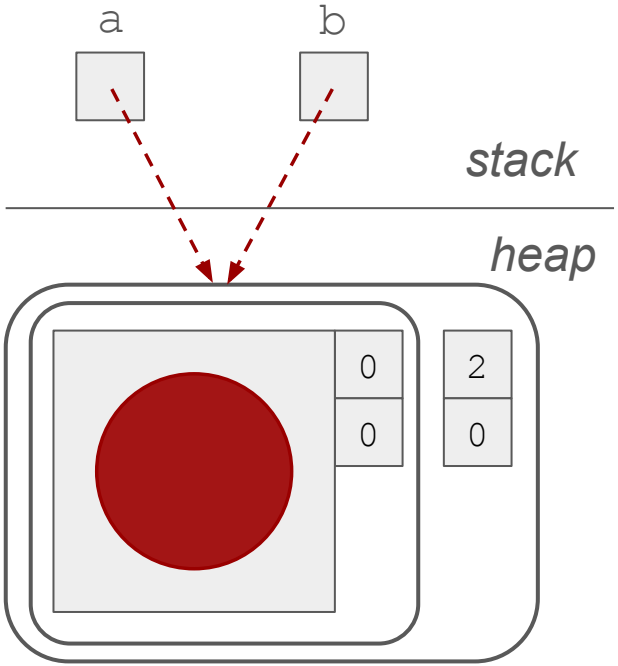
If we only have shared references to a type, but we need to *gain* mutable references from them, then it'll just result in a runtime penalty.

Designing a Graph

We just designed a `RefCell<T>`

Let's draw out the composed `Rc<RefCell<T>>`.

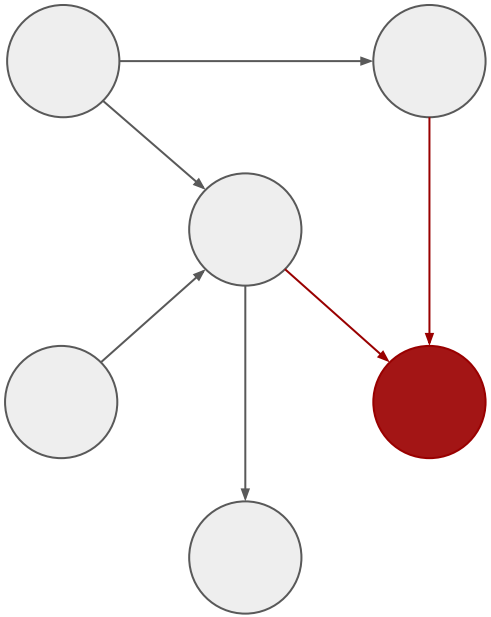
`Rc` allows us to have shared ownership, `RefCell` lets us safely mutate the underlying data.



Designing a Graph

Let's say we want (for whatever reason) to create a linked graph structure using the following:

```
struct Node<T> {
    my_data: T,
    neighbors: Vec<Rc<RefCell<Node>>>
}
```

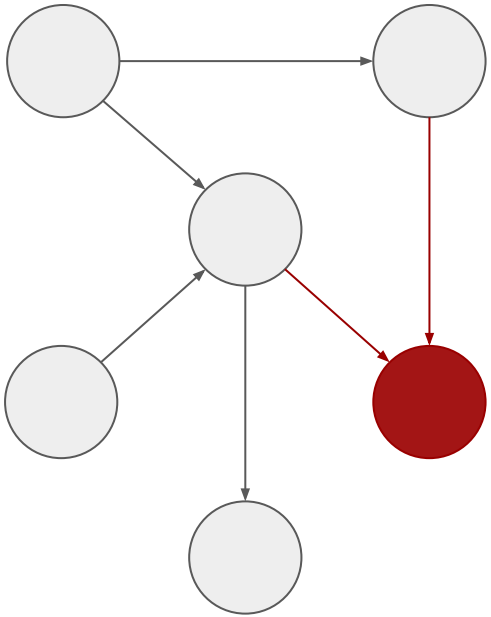


This fixes our multiple-owner problem, and let's us mutate nodes as necessary.

Designing a Graph

Let's say we want (for whatever reason) to create a linked graph structure using the following:

```
struct Node<T> {
    my_data: T,
    neighbors: Vec<Rc<RefCell<Node>>>
}
```



Just like `Rc`/`Arc`, there exist thread-safe versions of `RefCell<T>`, like `Mutex<T>` / `RwLock<T>`!

Send/Sync trait bounds will stop you from using a non-thread safe version when thread safety is required.

A Fun Question

`Rc<RefCell<T>>` and `Arc<Mutex<T>>` are really common patterns for multi-owner, mutable access to `T`.

But what would `RefCell<Rc<T>>` allow us to do?

A Fun Question

`Rc<RefCell<T>>` and `Arc<Mutex<T>>` are really common patterns for multi-owner, mutable access to `T`.

But what would `RefCell<Rc<T>>` allow us to do?

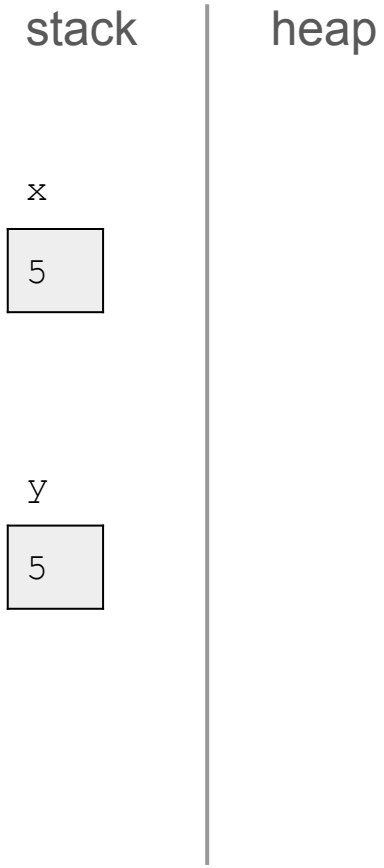
- You can swap the contained `Rc` by getting a mutable reference to it.
  - As in, change it to a completely different `Rc` pointer.
- Still can't mutate `T` though…
- `Rc` is kind of pointless, there would still only be one owner of the `RefCell`.

In general, chances are you *probably* don't want to use this.

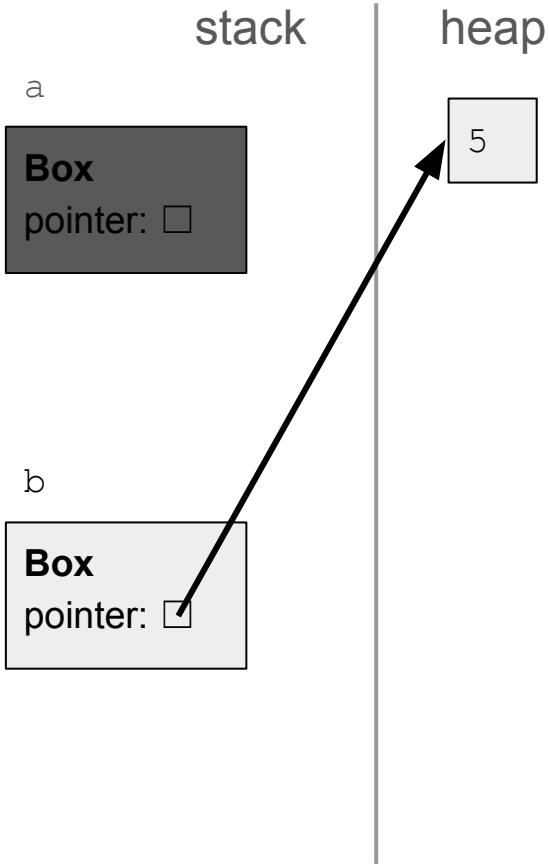
Do Worksheet Questions 1 and 2!

- 5 minutes
- Let me know if you’re confused - I’m here to help!

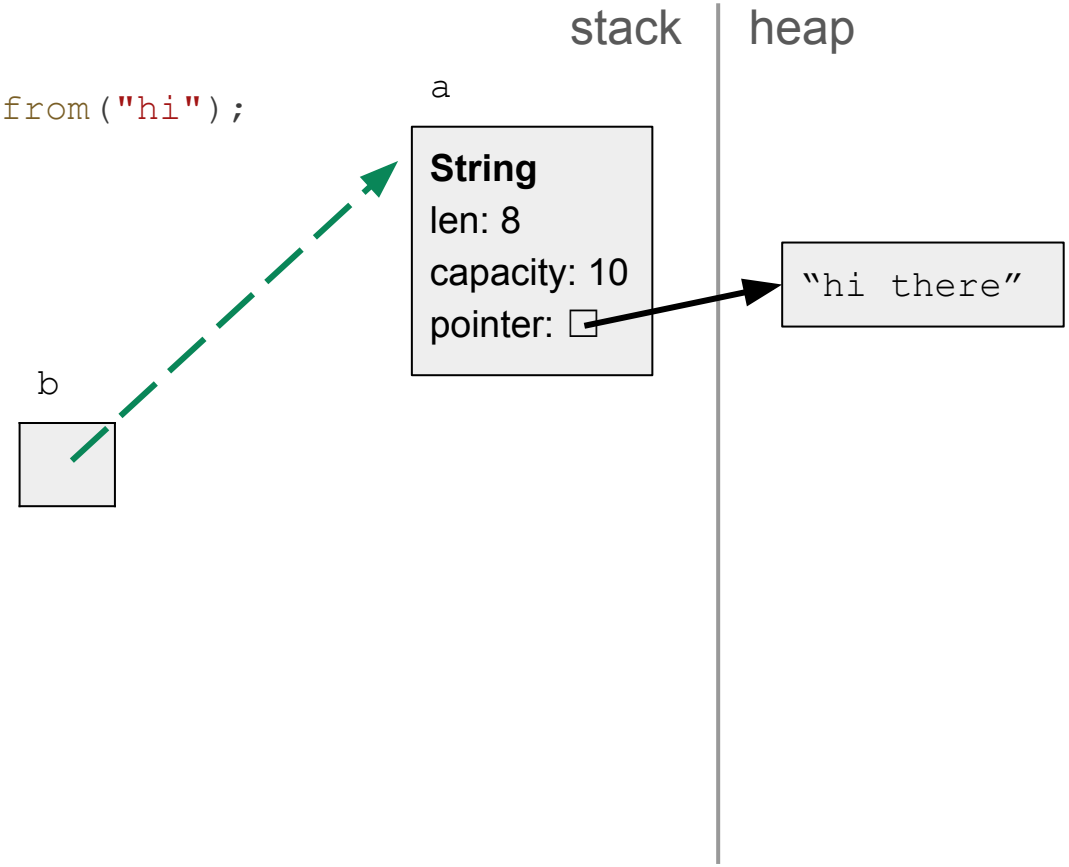
```rust
fn function_1() {
    let x: i32 = 5;
    let y: i32 = x;
    dbg!(x, y);
}
```



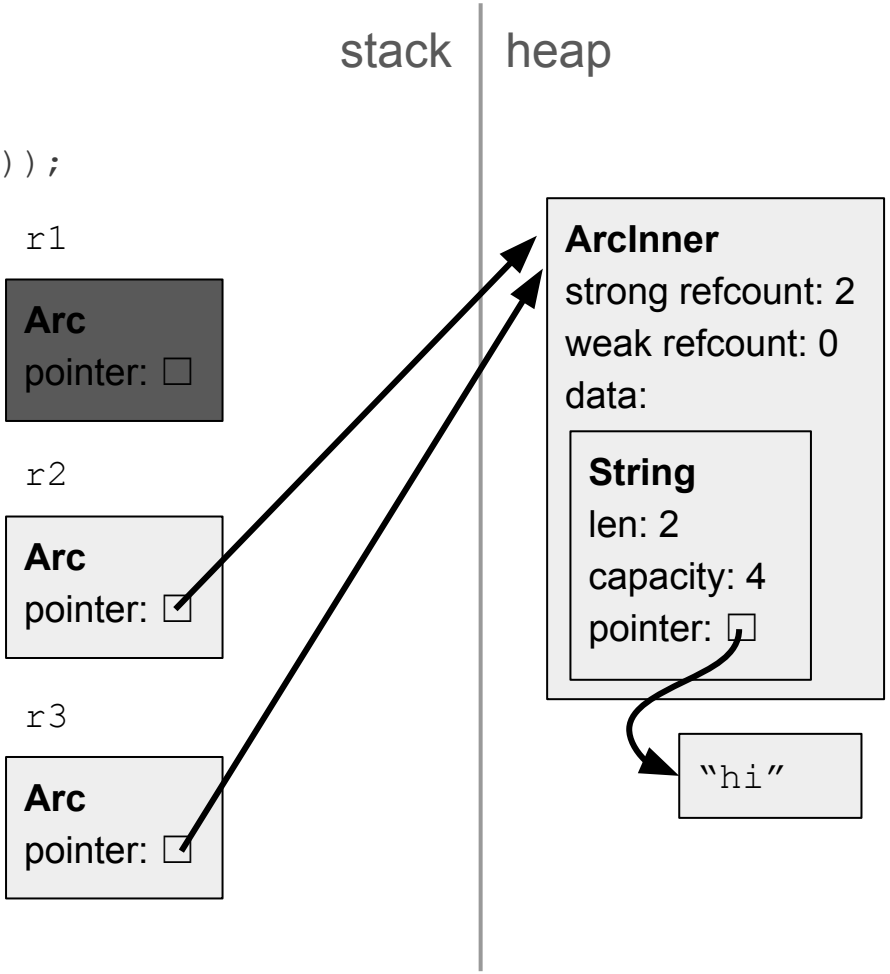
```rust
fn function_2() {
    let a: Box<i32> = Box::new(5);
    let b: Box<i32> = a;
    dbg!(b);
}
```



```rust
fn function_3() {
    let mut a: String = String::from("hi");
    let b: &mut String = &mut a;
    b.push_str(" there");
    dbg!(a);
}
```

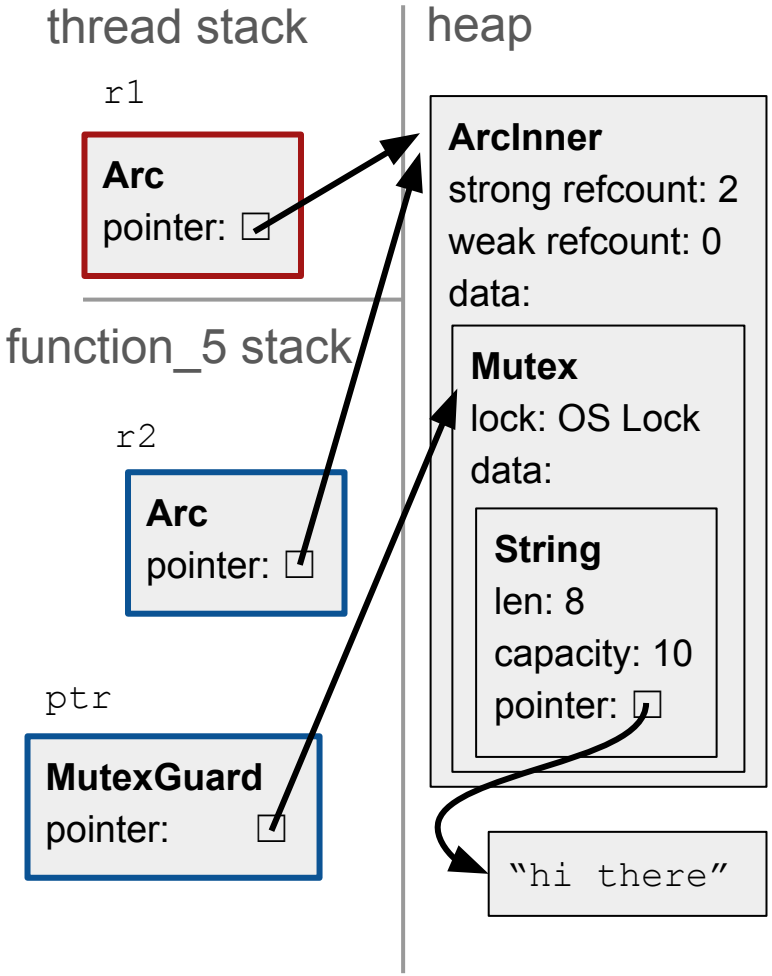


```rust
fn function_4() {
    let r1 = Arc::new(String::from("hi"));
    let r2 = r1.clone();
    let r3 = r2.clone();
    drop(r1);
    dbg!(r2, r3);
}
```



```rust
fn function_5() {
    let r1 = Arc::new(Mutex::new(String::from("hi")));
    let r2 = r1.clone();

    let _handle = thread::spawn(move || {
        let mut ptr: MutexGuard<String> = r1.lock().unwrap();
        ptr.push_str(" there");
    });
    {
        let ptr: MutexGuard<String> = r2.lock().unwrap();
        dbg!(ptr);
    }
    // `_handle` is dropped so the thread is detached
    // so it keeps running until the program exits
}
```



```rust
impl<T> Mutex<T> {
    fn lock(&self) -> Result<MutexGuard<T>, PoisonError<T>> {
        todo!()
    }
}
```

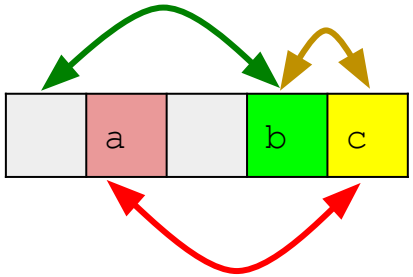
- Multiple threads must be able to have references to the same **Mutex**.
  - **&mut Mutex** wouldn't allow sharing, so they must be **&Mutex** (shared/immutable) references
  - Note: **Arc<Mutex<T>>** behaves like **&Mutex<T>**.
- **MutexGuard<T>** is like an **&mut T** (except it unlocks when dropped).
- **lock()** takes **&Mutex<T>** and essentially gives you **&mut T**.
- "Interior mutability".

Homework 6: Cuckoo HashMap

- You are given an implementation of a non-concurrent cuckoo hashmap.
  - The code we give has some unnecessary cloning. You're not required to fix it.
  - If you're up to the challenge though, it's not too difficult to get rid of unnecessary clones
- Your job is to make it supports *concurrent* inserts from multiple threads.

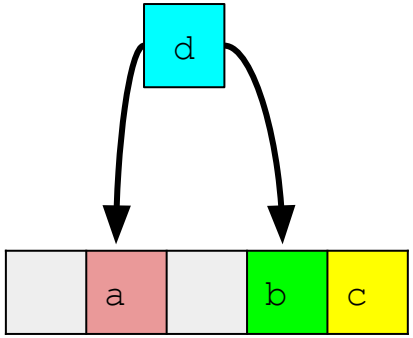
Homework 6: Cuckoo HashMap

- Backed by a Vec whose elements are buckets.
- Each key is hashed to 2 distinct buckets (using 2 hash fxs and a tiebreaker).
- Lookup/get just looks into both buckets to see if the key is there, O(1)



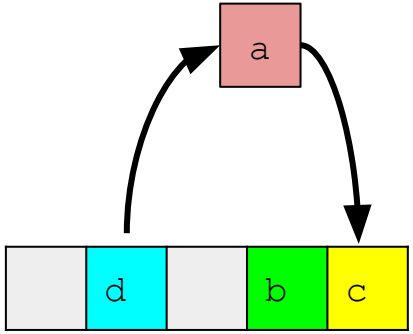
Cuckoo HashMap Insert

- Let's execute `insert(d)`
- Both buckets that `d` hashes to are full!!!
- Let put `d` in the place of `a`, and move `a` to another bucket.



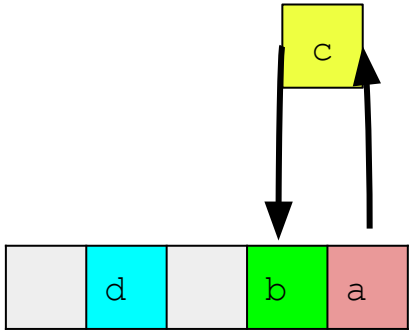
Cuckoo HashMap Insert

- But the other bucket of `a` is taken by `c`!
- Let's put `a` there, and move `c` to another bucket.



Cuckoo HashMap Insert

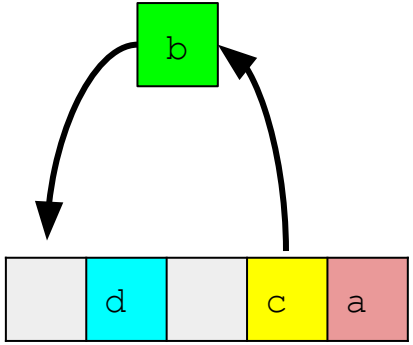
- But the other bucket of `c` is taken by `b`!
- Let's put `c` there, and move `b` to another bucket.



Cuckoo HashMap Insert

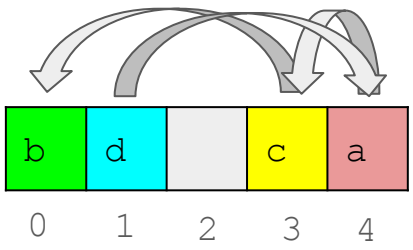
- The other bucket of `b` is free!

What can go wrong amidst this process?



Insertion Loops

- Before we insert, determine the “insertion path”.
- If the path doesn’t finish in some preset number of steps, assume there’s an endless loop.
  - If so, check if inserting to the key’s other bucket would also cause a loop.
- If both cause loops, double the number of buckets, and/or generate different hash functions.
  - In HW 6 you can return an error when resizing/rehashing would be necessary.




```
insertion_path:
[1, 4, 3, 0]
```

How Do We Make Cuckoo Support Simultaneous Inserts?

Take 3 minutes to do Question 3:

“How would you let multiple threads simultaneously modify different elements of the same **Vec**? Write the full type you would give to threads.”

What are the tradeoffs between the following:

- Arc<Mutex<Vec<T>>>
- Arc<Vec<Mutex<T>>>
- Vec<Arc<Mutex<T>>>

How Do We Make Cuckoo Support Simultaneous Inserts?

Take 3 minutes to do Question 3:

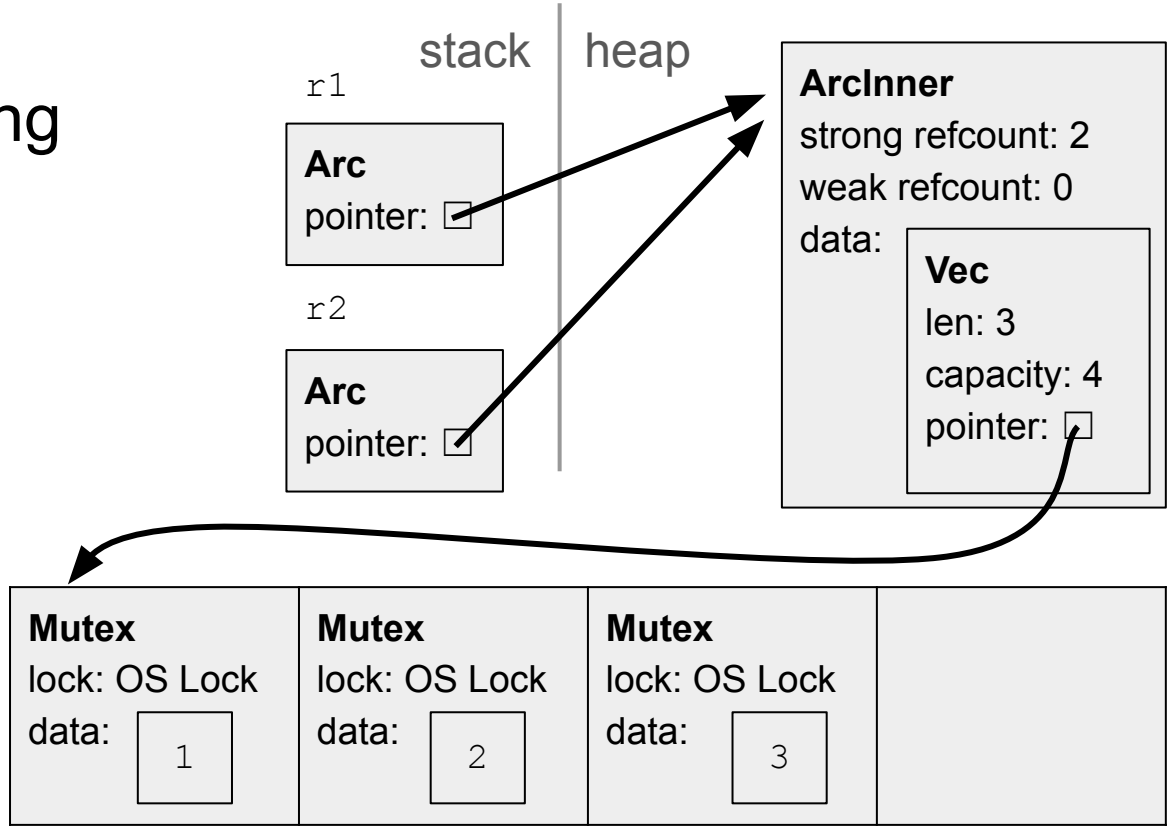
“How would you let multiple threads simultaneously modify different elements of the same **Vec**? Write the full type you would give to threads.”

What are the tradeoffs between the following:

- Arc<Mutex<Vec<T>>> → No parallel access to elements.
- Arc<Vec<Mutex<T>>> → Good! But some overhead in needing to lock often.
- Vec<Arc<Mutex<T>>> → Doesn’t allow shared ownership.

Fine-Grained Locking

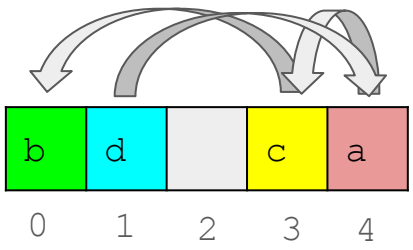
`Arc<Vec<Mutex<i32>>>`



More On Insertion

We want to find the insertion path for `insert(d)`

Once we find the path, we can then shift elements as necessary, *from right to left*!



`insertion_path:`

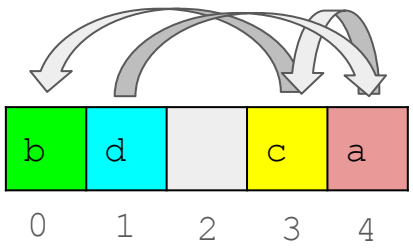
`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

Once we find the path, we can then shift elements as necessary, *from right to left*!

- *Why not left to right? What can go wrong?*



`insertion_path:`

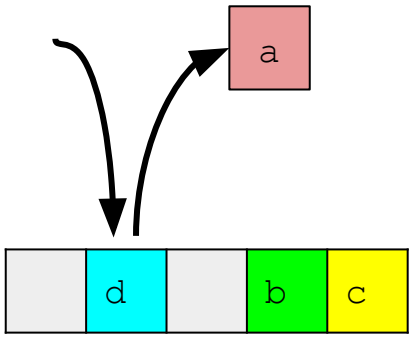
`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

Once we find the path, we can then shift elements as necessary, *from right to left*!

- *Why not left to right? What can go wrong?*



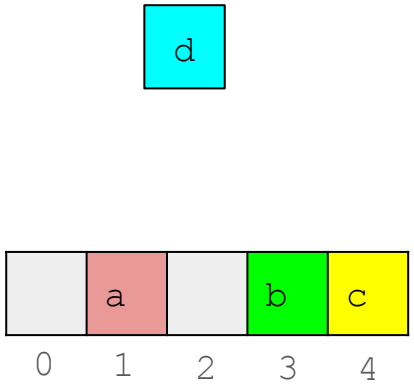
`insertion_path:`

`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

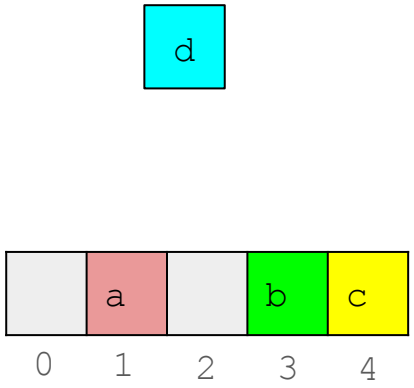


More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

- One lock at a time?



`insertion_path:`

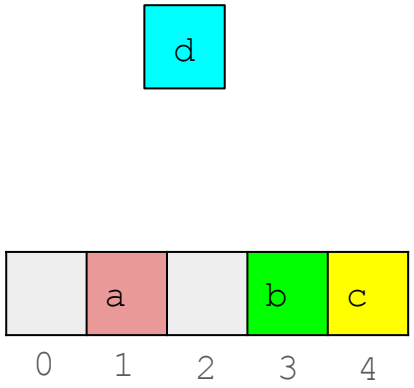
`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

- One lock at a time?
- Let’s consider moving index 3 to index 0.



`insertion_path:`

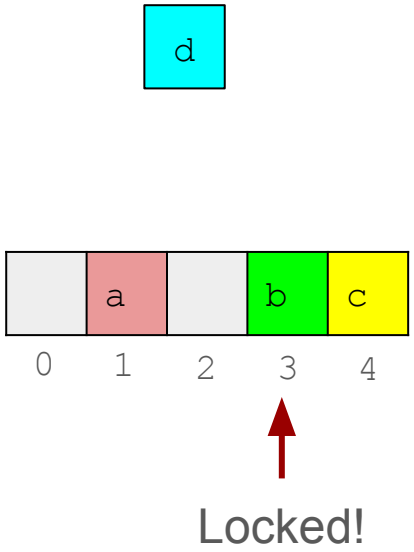
`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

- One lock at a time?
- Let’s consider moving index 3 to index 0.



`insertion_path:`

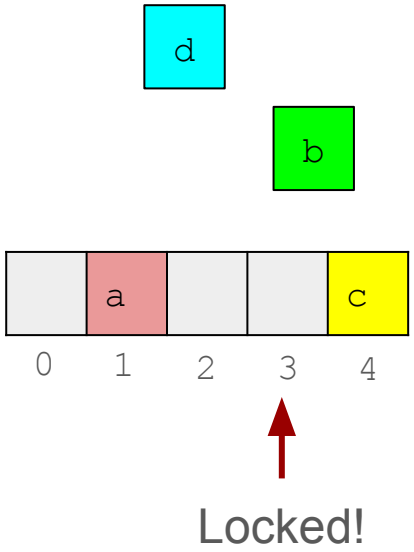
`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

- One lock at a time?
- Let's consider moving index 3 to index 0.



`insertion_path:`

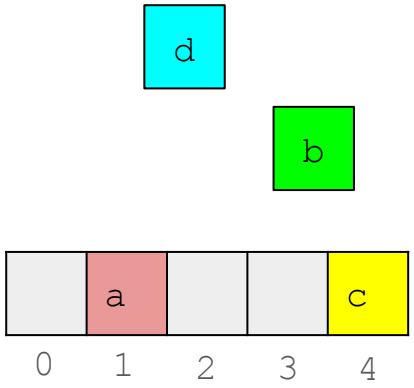
`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

- One lock at a time?
- Let’s consider moving index 3 to index 0.



`insertion_path:`

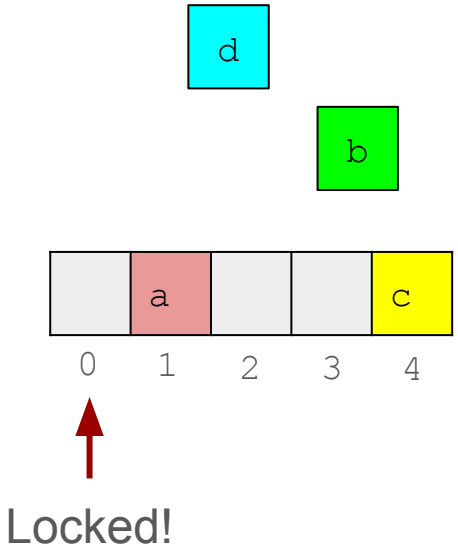
`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

- One lock at a time?
- Let’s consider moving index 3 to index 0.



`insertion_path:`

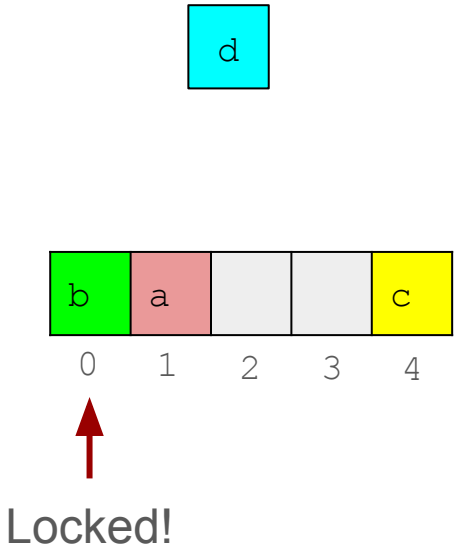
`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

- One lock at a time?
- Let’s consider moving index 3 to index 0.



`insertion_path:`

`[1, 4, 3, 0]`

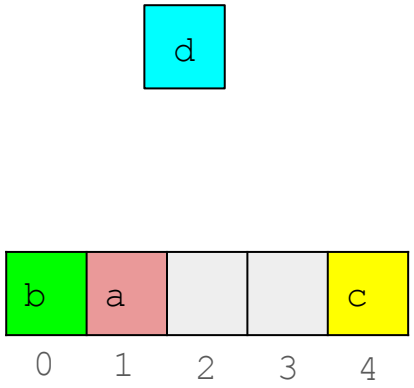
More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

- One lock at a time?
- Let's consider moving index 3 to index 0.

What's wrong with this *one-at-a-time* approach?



`insertion_path:`

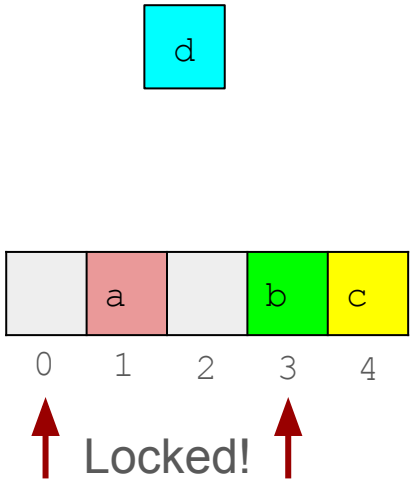
`[1, 4, 3, 0]`

More On Insertion

We want to find the insertion path for `insert(d)`

To access elements, how should we acquire / release locks?

- One lock at a time?
- Let's consider moving index 3 to index 0.



Acquire both bucket locks before moving any data!

- Note: this can lead to deadlock!

`insertion_path:`

`[1, 4, 3, 0]`

Loom

- Exhaustively tests all concurrent interleavings to reproduce rare bugs.
- Varies order in which threads pass synchronization points such as lock().
- Tries to skip equivalent executions for efficiency.
- Requires Loom-specific versions of Mutex, thread, etc.
    - Can swap out on tests using conditional compilation:

```rust
#[cfg(loom)]
use loom::sync;
#[cfg(not(loom))]
use std::sync;
```

```
RUSTFLAGS="--cfg loom" cargo test --test loom_tests --release
```